# Übungsblatt 5: Ereignisse, Queues und Speicherbesitz

**Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

## Aufgabe 1: Eine FIFO durch den Ring verfolgen (8 Punkte)

Ein Ring mit vier Plätzen startet leer; `head = tail = count = 0`. Head bezeichnet den nächsten Get-Platz, Tail den nächsten Put-Platz, beide modulo vier. Ein erfolgreicher Put erhöht Count, ein erfolgreicher Get verringert ihn.

Verfolgen Sie: Put 7, Get, **Try-Get mit Timeout null**, Put 9, Put 12, Put 15, Put 18, Get, Put 21.

1. Geben Sie nach jedem Schritt Head, Tail, Count und die logisch gültige FIFO-Folge an. (4 Punkte)
2. Was liefern die beiden erfolgreichen Gets? Wieso bedeutet Head gleich Tail nicht eindeutig leer? (2 Punkte)
3. Ein weiterer Try-Put mit Timeout null trifft auf den Endzustand. Wird er angenommen, und ändern sich Head, Tail oder Count? Darf nach dem Try-Get aus Schritt 3 ein alter Inhalt des Zielobjekts als neue Nachricht verarbeitet werden? (1 Punkt)
4. Geben Sie den physischen Arrayinhalt an den Indizes 0 bis 3 nach der angegebenen Folge an. Lesen Sie daraus die logische FIFO-Folge ab Head ab. (1 Punkt)

## Aufgabe 2: Ankunft, Bedienung und Überlauf (8 Punkte)

Eine FIFO hat fünf Plätze und startet leer. Bei 2, 4, …, 24 ms wird je eine Nachricht mit Sequenz 1, 2, …, 12 angeboten. Gets bei 8, 16 und 24 ms entnehmen je ein Element. Bei Gleichzeitigkeit gilt **Get vor Put**. API-Zeit null, bei Vollbelegung wird die neue Nachricht verworfen.

1. Bestimmen Sie den ersten Fehl-Put und alle verworfenen Sequenzen. (2 Punkte)
2. Nennen Sie entnommene Werte, die Zahl erfolgreicher Puts und die Restfolge. Verwenden Sie $N_{\text{ang}}$ für Angebote, $N_{\text{ok}}$ für erfolgreiche Puts, $N_{\text{drop}}$ für verworfene Angebote, $N_{\text{get}}$ für erfolgreiche Entnahmen und $N_{\text{rest}}$ für verbliebene Nachrichten. Prüfen Sie die Bilanzen $N_{\text{ang}} = N_{\text{ok}} + N_{\text{drop}}$ und $N_{\text{ok}} = N_{\text{get}} + N_{\text{rest}}$. (2 Punkte)
3. Welche Mindestkapazität genügt für dieses Intervall ohne Verluste? Betrachten Sie dafür einen verlustfreien Referenzlauf mit denselben Angeboten und Get-Zeitpunkten. Kann ein endlicher Puffer bei dauerhaft denselben Raten jeden Verlust verhindern? (2 Punkte)
4. Zwei neue Anwendungen haben eine Quelle, die nicht auf die Software wartet: Ein Display benötigt nur den jüngsten gültigen Wert; ein Ereignisarchiv verlangt jeden erfassten Datensatz. Welche Überlaststrategie passt jeweils? Begründen Sie, wann das Ersetzen älterer Werte zulässig ist. (2 Punkte)

## Aufgabe 3: Datensatz oder Pufferadresse? (8 Punkte)

1. Eine Wertqueue kopiert `{seq=6, value=37}`. Der Sender setzt seine lokale Variable danach auf 88. Was erhält der Consumer? Was ändert sich bei einer Queue reiner Pointer? (2 Punkte)
2. Sechs Pointerplätze zu vier Byte stehen drei Bildpuffern zu je 48 KiB gegenüber. Einer wird gelesen, zwei sind eingereiht. Wie viele unabhängige neue Bilder können sofort befüllt werden? Berechnen Sie Pointer-Nutzspeicher und Bildspeicher getrennt. (3 Punkte)
3. Wer besitzt einen Puffer nach fehlgeschlagenem Pointer-Put? Warum genügen `static` oder zwei Get-Consumer nicht für „beide Empfänger bekommen jedes Bild“? (3 Punkte)

## Aufgabe 4: Ein Ereignis bis zur Workerreaktion (8 Punkte)

1. Vier NVIC-Prioritätsbits, passende Gruppierung, logische Kernelaufrufgrenze sechs: Bewerten Sie Queue Put mit Timeout null aus IRQ-Priorität fünf und sieben. Welche Zahl bezeichnet die höhere IRQ-Dringlichkeit? Darf ein erlaubter IRQ einen Mutex erwerben? (2 Punkte)
2. Gemeinsame Modelluhr in µs: Ereignis 40, ISR-Eintritt 45, Worker Ready 52, ISR-Ende 58, Workerstart 65, Reaktion abgeschlossen 79. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Warten und Gesamtantwort. Prüfen Sie $D = 35\,\mu s$ und vermeiden Sie Doppelzählung. (3 Punkte)
3. Unterscheiden Sie Tasterprellen, Zusammenfassung mehrerer Hinweise in einem Bit und Drop einer bereits erfassten Queue-Nachricht. (1 Punkt)
4. Ein Messregister kann vor dem Workerstart bereits einen neuen Wert enthalten. Der bei ISR-Eintritt vorliegende Wert soll für eine spätere Auswertung erhalten bleiben. Welche Arbeit muss kurz in der ISR erfolgen, welche kann der Worker übernehmen? Genügt ein Weckbit mit späterem Registerlesen? (2 Punkte)

**Implementierungshinweis ohne Punkte:** Im hier vorgegebenen Vier-Bit-Modell entspricht die logische Grenze sechs dem Registerwert $6 \cdot 2^{8-4} = 96 = 0x60$. Der HAL-Prioritätssetter erhält den logischen Wert sechs.

## Aufgabe 5: Zwei Queue-Einträge, derselbe Puffer (4 Punkte)

Eine Queue nimmt zweimal den Pointer auf dasselbe statische Array an. Beim ersten Put enthält es `[3,8]`, vor dem zweiten Put wird es auf `[5,13]` geändert. Der Consumer liest erst danach beide Einträge.

1. Welche Inhalte sieht der Consumer? Begründen Sie Ihre Antwort. (2 Punkte)
2. Beschreiben Sie eine korrekte Zwei-Puffer-Übergabe, die beide ursprünglichen Stände erhält. Wann darf der Producer die Puffer wiederverwenden? (2 Punkte)

## Aufgabe 6: Tasterereignisse an einen LED-Worker übergeben (12 Punkte)

**Programmieren, etwa 35–45 Minuten.** Die CM4-Basis `lab05_button.c/.h` und `button_student.c` enthält eine FIFO mit drei Plätzen und genau einen Worker. Nur der eingebaute **B1/PC13** erzeugt Ereignisse; **LD1/PB0** zeigt bearbeitete Betätigungen. FreeRTOS/CMSIS-RTOS2 läuft mit 1000 Ticks/s. Der vorhandene EXTI-Callback übergibt seine Pinmaske mit `Lab05_ButtonIRQ(pin)`; dieser geprüfte Einstieg ruft für B1 Ihre Funktion auf.

1. Implementieren Sie `Lab05_QueueButton()` für den IRQ: Datensatz `Lab05_Press` aus fortlaufender Sequenz und `osKernelGetTickCount()` bilden, Try-Put mit Nachrichtenpriorität null und Timeout null. Zählen Sie Annahmen, volle-Queue-Drops und sonstige Fehler getrennt. In der ISR weder warten noch LEDs verzögert bedienen. (3 Punkte)
2. Implementieren Sie `Lab05_PulseWorker(argument)` mit blockierendem Get. Prüfen Sie den Rückgabestatus: Erfolg bedeutet `status == osOK`, nicht `if (status)`. Verwenden Sie dazu `Lab05_ButtonCheck(status)`. Der erste Datensatz zählt; weitere mit weniger als 40 Tickwerten Abstand zum letzten vom Worker als Betätigung gewerteten Ereignis werden als Prellen ignoriert. Vergleichen Sie die erfassten Ereigniszeitpunkte, nicht die spätere Bearbeitungszeit. (4 Punkte)
3. Für jeden angenommenen Tastendruck LED einschalten, `osDelay(60U)`, ausschalten und `osDelay(100U)`. Prüfen Sie API-Fehler über `Lab05_ButtonCheck(status)`; Queue-Vollbelegung im IRQ wird gezählt und nicht als normaler Erfolg behandelt. (2 Punkte)
4. Prüfen Sie einzelne und schnelle Betätigungen. Zählen elektrische Flanken, erreichte Callbacks, erfolgreiche Puts und begonnene LED-Pulse notwendigerweise dasselbe? Interpretieren Sie die Diagnosewerte erst an einem konsistenten Prüfpunkt. (3 Punkte)

**Vorbereitete Basis:** Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

**Quellen:** Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.